



RETRO
STORE

PROYECTO E-COMMERCE



MIGUEL BENJUMEA HIDALGO



RONNARD BENÍTEZ RECALDE



JUAN JIMÉNEZ NIETO



DAVID GUTIÉRREZ ORTIZ



Profesor

MARTIN RIVERO

CURSO 25 | 26

FECHA: 20 DE FEBRERO, 2026

DESARROLLADO POR: AV GAMES

0. NOTA SOBRE LA DOCUMENTACIÓN

Este documento es una versión resumida de la documentación del proyecto, preparada específicamente para la entrega y evaluación.

Aquí se incluyen los apartados más relevantes: descripción general, requisitos, diagramas, base de datos, rutas e instalación. Si se desea ampliar información o consultar explicaciones más detalladas (casos de uso completos, decisiones técnicas, etc.), se puede revisar el archivo **“Documentación_Detallada.md”**, disponible en la misma carpeta del proyecto.

1. DESCRIPCIÓN DEL PROYECTO

AVGames (Retro Store) es una plataforma e-commerce de videojuegos retro con estética synthwave/80s. Permite a los usuarios:

- Navegar por un catálogo de juegos
- Añadir productos al carrito
- Realizar un checkout con pago simulado
- Descargar archivos ZIP asociados
- Gestionar su perfil y nivel de usuario

Incluye un **panel de administración completo** para gestionar productos, categorías, usuarios y estadísticas. La arquitectura del sistema está basada en el patrón **MVC**, con una separación clara entre el backend desarrollado en Laravel y el frontend tipo SPA gestionado mediante Inertia.js.

1.2 STACK TECNOLÓGICO

- **Backend:** Laravel 11 (PHP 8.2+)
- **Frontend:** React 18 + TypeScript (Inertia.js)
- **Base de Datos:** MySQL 8.x (motor InnoDB, claves foráneas activadas)
Durante el desarrollo inicial se utilizó SQLite para pruebas rápidas, pero la versión final entregada funciona sobre MySQL para cumplir los requisitos de producción y consistencia relacional.
- **Autenticación:** Laravel Fortify (login, registro, 2FA)
- **Estilos:** Tailwind CSS (tema personalizado)
- **Bundler:** Vite

2. REQUISITOS FUNCIONALES

Funcionalidades para Usuarios

- Registro e inicio de sesión
- Navegación y búsqueda en catálogo
- Filtros por categoría
- Añadir al carrito (requiere archivo ZIP activo)
- Checkout con:
“Pedido gratuito (auto-completado)” y “Pedido de pago (pago simulado)”
- Descarga protegida de juegos
- Historial de pedidos
- Sistema de niveles basado en descargas

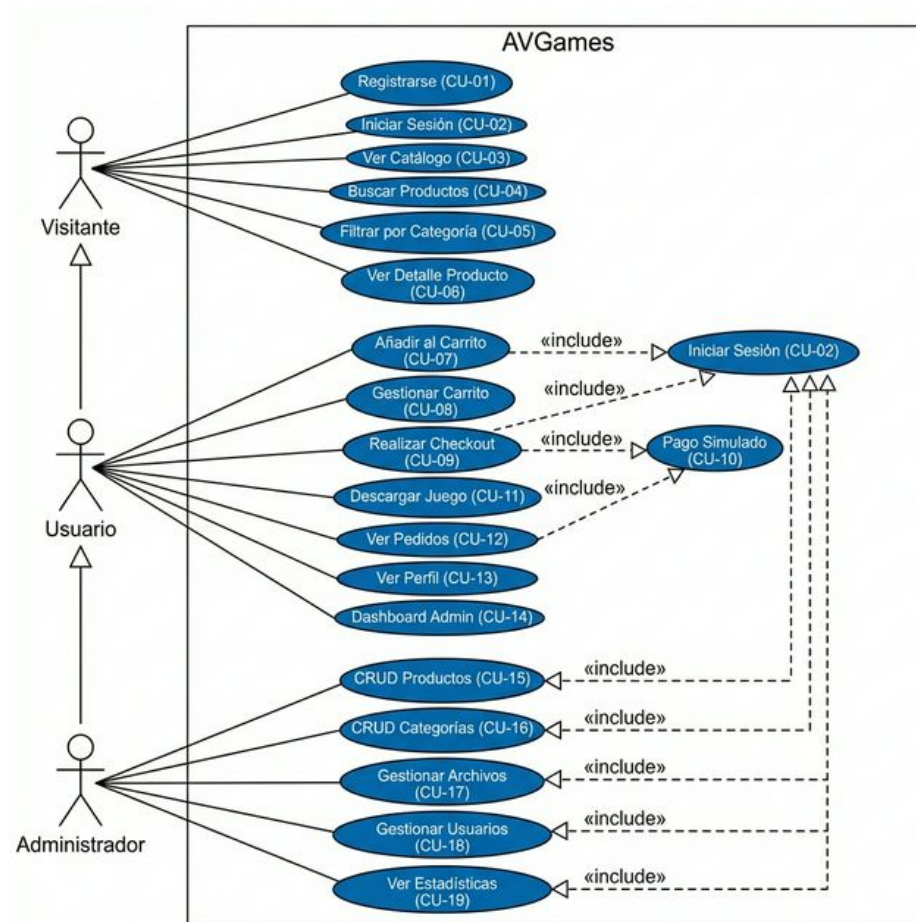
Funcionalidades para Administradores

- Dashboard con estadísticas
- CRUD completo de productos
- CRUD completo de categorías
- Gestión de archivos descargables (ZIP)
- Gestión de usuarios (activar, suspender, banear, promover a admin)

3. REQUISITOS NO FUNCIONALES

- Seguridad mediante middleware auth + verified + admin
- Descargas protegidas por validación adicional en backend
- Arquitectura MVC clara (Laravel)
- Integridad mediante claves foráneas y cascade
- SPA con Inertia.js (sin recargas completas)
- Sistema extensible preparado para múltiples métodos de pago

4. DIAGRAMA DE CASOS DE USO



Relaciones «include»

- Checkout incluye Login
- Pago Simulado incluye Confirmación de Pedido
- Descargar Juego incluye Login
- Todas las rutas Admin incluyen Login + Rol Admin

Casos de Uso Clave (resumen textual)

Login

Permite autenticación del usuario. Si no está autenticado, se redirige automáticamente.

Checkout

Crea un pedido:

- Si total = 0 → se marca como completed automáticamente.
- Si total > 0 → pasa a pantalla de pago simulado.

Pago Simulado

Formulario ficticio según método (tarjeta, PayPal, transferencia).

Al confirmar → Order pasa a completed.

Descargar Juego

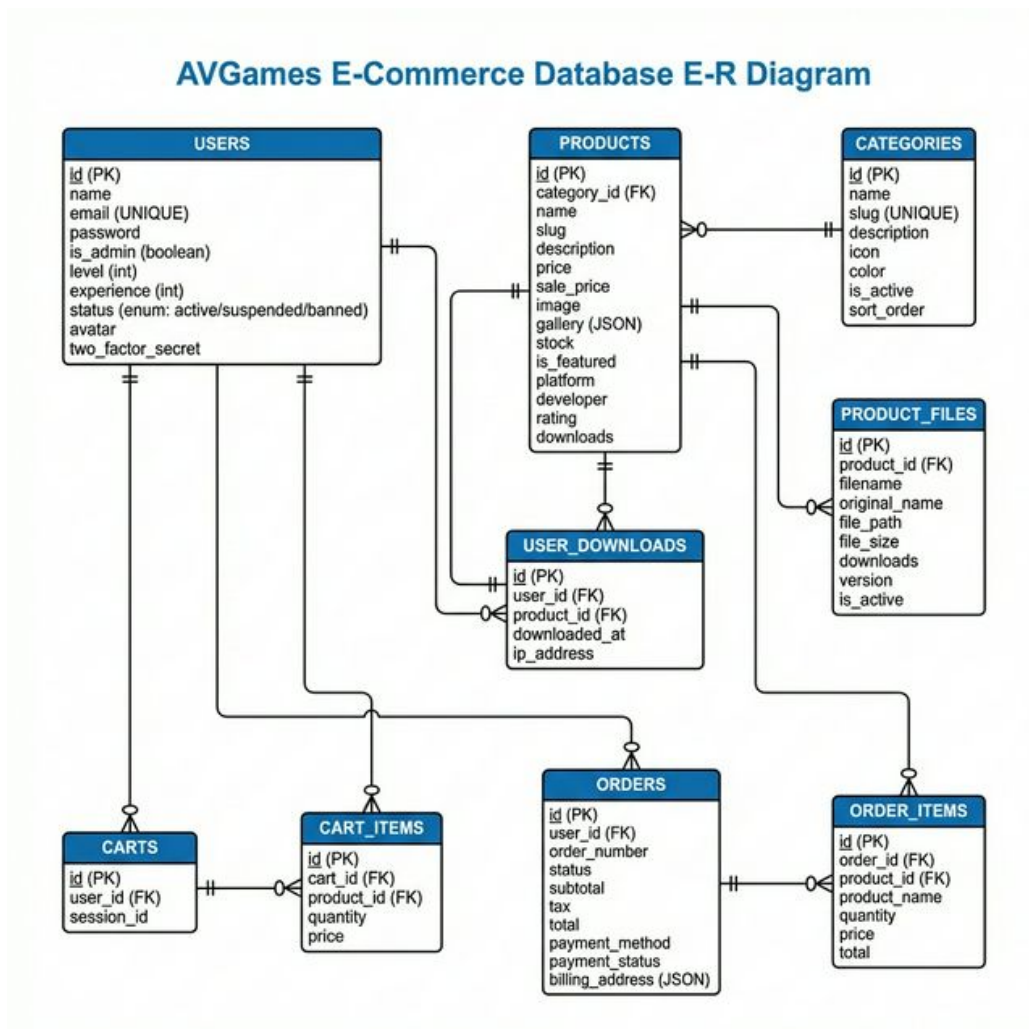
Verifica:

- Usuario autenticado
- Pedido completado (si es de pago)
- Archivo activo

Gestionar Usuarios (Admin)

Permite modificar estado, nivel y rol de usuario.

5. DIAGRAMA E-R



Las relaciones están implementadas mediante claves foráneas reales en MySQL con integridad referencial y eliminación en cascada donde corresponde.

Entidad	Relación	Entidad	Tipo
User	hasMany	Order	1:N
User	hasOne	Cart	1:1
User	belongsToMany	Product	M:N
Category	hasMany	Product	1:N
Product	hasMany	ProductFile	1:N
Order	hasMany	OrderItem	1:N
Cart	hasMany	CartItem	1:N

6. ESTRUCTURA BASE DE DATOS

users

- id
- name
- email
- password
- is_admin
- level
- status

categories

- id
- name
- slug
- icon
- color
- is_active

products

- id
- category_id
- name
- slug
- price
- sale_price
- is_active
- downloads

product_files

- id
- product_id
- file_path
- version
- is_active

orders

- id
- user_id
- status
- total
- payment_method
- payment_status

order_items

- order_id
- product_id
- quantity
- price

7. RUTAS PRINCIPALES

Públicas

- /
- /library
- /product/{slug}

Carrito

- /cart
- /cart/add/{product}

Protegidas

- /checkout
- /payment/{order}
- /orders
- /download/game/{productFile}

Admin

- /admin
- /admin/products
- /admin/categories
- /admin/users
- /admin/statistics

Todas las rutas protegidas implementan middleware auth, verified y validaciones adicionales a nivel de controlador para aplicar un modelo de seguridad de doble capa.

8. INSTRUCCIONES DE INSTALACIÓN

```
composer install
npm install
cp .env.example .env
php artisan key:generate
php artisan migrate:fresh --seed
php artisan storage:link
npm run build
php artisan serve
```

Credenciales

Admin:

admin@avgames.com

password

Usuario:

test@example.com

password

9. SISTEMA DE NIVELES

Nivel calculado por descargas:

$\text{Nivel} = \text{floor}(\text{descargas} / 5) + 1$

El nivel se recalcula dinámicamente tras cada descarga registrada en la tabla `user_downloads`, sin almacenarse como valor fijo permanente

10. Rúbrica (Compacta)

Criterio	Estado
Migraciones y Eloquent	✓
CRUD completo	✓
Checkout funcional	✓
Pago simulado	✓
Descargas protegidas	✓
Panel Admin	✓
Documentación	✓